Agile & DevOps Practical: Steps and Code Lines

Experiments 1, 2, 3, 4 and 7

One cheat sheet for the practical: only the steps and the exact commands / code for each experiment, in performance order. Worked example throughout: Food Ordering App, Jira space key FOOD, repo Jira-Try.

Experiment 1: Jira Lifecycle with DevOps Integration

1. **Create the space.** Create a space: Scrum template, team-managed. Name it (e.g. "Food Ordering App") and give it a key (e.g. FOOD). The key prefixes every work item.
2. **Create work items.** Create an Epic (e.g. "User Management"), then Stories and Tasks (e.g. Story "As a User, I want to Login so I can order Food", Task "Design Login UI and Login API", Story "As a User, I want to Register so I can use the App", Task "Design Register UI and API").
3. **Parent items to the Epic.** Set the Parent field of every story and task to the Epic. The epic then shows as a pill on its children, and the left Epic panel filters the backlog.
4. **Run the sprint.** Backlog -> Start sprint -> Board: drag items To Do -> In Progress -> Done (at least one item to Done). Then Complete sprint: Done items leave the backlog, unfinished items return to it.
5. **Connect GitHub.** Left sidebar -> Apps -> Explore more apps -> install "GitHub for Jira" -> connect GitHub account -> select repositories.
6. **Link code to a work item.** Clone, change a file, commit with the work-item key first in the message, push:

```
git clone https://github.com/Infindev7/Jira-Try.git
git add .
git commit -m "FOOD-8 Designing Login and API"
git push
```

7. **Verify the link.** Open the work-item card in Jira -> Development section: the pushed commit appears there, linked to the card.
8. **Branch, pull request, merge.** Branch with the key in its name, commit, push, open a pull request, merge it, delete the branch, sync local main:

```
git checkout -b feature/FOOD-9-register
git add README.md
git commit -m "FOOD-9 add register page info"
git push -u origin feature/FOOD-9-register
```

On GitHub: open the pull request -> Merge pull request -> Confirm merge -> Delete branch. Then:

```
git checkout main
git pull
```

9. **Automation flow.** Space settings -> Automation -> Create flow -> Trigger: Pull request merged -> Action: Transition work item -> Destination status: Done -> name the flow (e.g. "PR merged -> Done") -> Save and enable.

Note: In the action's Destination status dropdown, explicitly select Done. Do NOT leave the default "Copy from trigger work item" (it only copies the existing status, so nothing changes).

10. **Test the flow.** Test it: merge a pull request whose branch or title carries a work-item key. The card moves to Done by itself.

Note: Releases / Versions: the toggle does not appear in Space settings -> Features on the Free plan for team-managed spaces. Viva line: a version groups finished work for delivery; marking it Released means it shipped; the completed sprint plus Done items are the release scope.

Experiment 2: Git

11. **Get the repo.** Clone a repository (or start one locally):

```
git clone https://github.com/Infindev7/Jira-Try.git
git init
git remote add origin https://github.com/Infindev7/Jira-Try.git
```

12. **Inspect.** Check state and history:

```
git status
git log --oneline
```

13. **Basic workflow.** Make a change, stage it, commit it, push it:

```
git add .
git commit -m "FOOD-8 Designing Login and API"
git push
```

14. **Branch.** Create a branch, work on it, push it (first push needs -u to link it):

```
git checkout -b feature/FOOD-9-register
git add README.md
git commit -m "FOOD-9 add register page info"
git push -u origin feature/FOOD-9-register
```

15. **Pull request and merge.** Open a pull request on GitHub, merge it, delete the branch, then sync local main:

```
git checkout main
```

```
git pull
```

16. **Merge conflict.** Merge conflict: it happens when the same line is changed two different ways on two branches. Git stops the merge and marks the lines in the file:

```
<<<<<<< HEAD
your version of the line
=======
other branch version of the line
>>>>>>> branch-name
```

Edit the file to keep one version (or a combination), delete all marker lines, then finish the merge:

```
git add <file>
git commit
```

Experiment 3: CI Pipeline (Jenkins and GitHub Actions)

Version A: Jenkins

17. **Check Java.** Check Java (Jenkins runs on Java 17, 21 or 25 with the current LTS):

```
java -version
```

18. **Start Jenkins.** Download the latest LTS jenkins.war, then run it (leave that terminal window running; Jenkins lives in it):

```
java -jar jenkins.war
```

Open http://localhost:8080. Unlock with the initial admin password printed in the terminal, choose Install suggested plugins, create the admin user, keep the URL as http://localhost:8080/ on the Instance Configuration screen, and Save and Finish to reach the dashboard.

19. **Create the Pipeline job.** Dashboard -> New Item -> name it (e.g. food-ordering-ci) -> Pipeline -> OK. In the Pipeline section: Definition = Pipeline script from SCM; SCM = Git; Repository URL = https://github.com/Infindev7/Jira-Try.git; Branch Specifier = */main; Script Path = Jenkinsfile. Save.

*Note: Branch Specifier defaults to */master. Change it to */main, or the build fails looking for a branch that does not exist. Public repo, so no credentials needed.*

20. **Jenkinsfile.** Add this Jenkinsfile to the repository root and push it:

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
```

```
            steps {
                echo 'Code checked out from GitHub'
            }
        }
        stage('Build') {
            steps {
                echo 'Building the Food Ordering App'
            }
        }
        stage('Test') {
            steps {
                echo 'All tests passed'
            }
        }
    }
}
git add Jenkinsfile
git commit -m "add Jenkins pipeline"
git push
```

21. **Build.** On the job page, click Build Now, open the build, and check Console Output. Expected result: all three stages green, ending in Finished: SUCCESS.

Note: Console lines "Selected Git installation does not exist. Using Default" and "The recommended git tool is: NONE" are harmless warnings, not errors.

Version B: GitHub Actions

22. **Create the workflow file.** In the repository, create the workflow file .github/workflows/ci.yml:

```
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run a check
        run: echo "CI pipeline running"
```

23. **Push.** Commit and push the file. The push itself triggers the workflow (on: [push]):

```
git add .github/workflows/ci.yml
```

```
git commit -m "add CI workflow"
git push
```

24. **Watch it run.** On GitHub, open the repository -> Actions tab. The CI run appears there; a green tick means the pipeline passed. Click the run to see the checkout and check steps in its log.

Note: No server or installation is needed for this version: GitHub's runners execute the workflow on every push.

Experiment 4: Docker (Two Versions)

Version A: Faculty version (ubuntu lifecycle)

25. **Pull the image.** Check Docker and pull the ubuntu image:

```
docker --version
docker pull ubuntu
docker images
```

26. **Run the container.** Run ubuntu interactively (your prompt changes to root@... inside the container):

```
docker run -it ubuntu /bin/bash
cat /etc/os-release
```

27. **Inspect from outside.** From a second command prompt, list containers and note the CONTAINER ID:

```
docker ps
docker container ls -a
```

28. **Stop and remove.** Stop the container, then remove it (stop first: Docker will not remove a running container without forcing it):

```
docker container stop <container-id>
docker container rm <container-id>
docker container ls -a
```

29. **Remove the image.** Remove the image (note the IMAGE ID from docker images; it is different from the container ID), verify it is gone, then run ubuntu once more:

```
docker images
docker image rm <image-id>
docker images
docker run -it ubuntu /bin/bash
```

Note: The last run re-downloads ubuntu automatically, since the image was just deleted. That is expected, not an error.

Version B: Dockerfile build (nginx)

30. **App page.** Create index.html in the project folder:

```
<!DOCTYPE html>
<html>
<head><title>Food Ordering App</title></head>
<body>
  <h1>Food Ordering App</h1>
  <p>Login and Register are live. Order your food!</p>
</body>
</html>
```

31. **Dockerfile.** Create the Dockerfile in the same folder (remember: Base, Place, Port):

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

32. **Build.** Build the image (-t names it; the dot means "build from this folder"):

```
docker build -t food-ordering-app .
```

33. **Run.** Run it detached, mapping host port 8080 to container port 80, and verify:

```
docker run -d -p 8080:80 food-ordering-app
docker ps
```

Open http://localhost:8080 in the browser: the Food Ordering App page appears.
Cleanup, as in Version A:

```
docker stop <container-id>
docker rm <container-id>
```

Experiment 7: AWS Lambda (Serverless Function)

34. **Create the function.** AWS Console -> search Lambda -> Create function -> Author from scratch.
35. **Configure.** Function name: food-ordering-hello. Runtime: the Python 3.x version offered. Leave architecture and the execution role on their defaults ("Create a new role with basic Lambda permissions"). Click Create function.

Note: Functions live in one region: note the region shown top-right of the console; that is where the function will be when you look for it later.

36. **Code.** In the code editor, write the handler:

```
import json

def lambda_handler(event, context):
```

```
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Food Ordering App!')
    }
```

37. **Deploy and test.** Click Deploy, then Test (create a test event; the default event is fine). The execution result shows the statusCode and body, and the logs go to CloudWatch.

Note: Serverless point for viva: the function runs only when invoked; there is no server to provision or manage, and you pay only for the invocations.